

Práctica 3: Context Switch

Mecanismos de concurrencia y planificación en un RTOS de misión crítica para marcapasos VVI

1 Objetivos

1.1 Objetivo general

Analizar, instrumentar y validar el mecanismo de cambio de contexto (*context switch*) y el comportamiento del planificador apropiativo (*preemptive scheduler*) en FreeRTOS sobre la arquitectura dual-core del ESP32, evaluando el impacto de la sobrecarga temporal (*overhead*) y la sincronización inter-tareas en la latencia del marcapasos VVI.

1.2 Objetivos específicos

- Instrumentar el guardado y restauración de registros en la pila (*stack frame*) durante la conmutación entre tareas con distintas prioridades utilizando los ganchos (*trace hooks*) y temporizadores de alta resolución de FreeRTOS.
- Cuantificar la latencia de conmutación de contexto frente a eventos asíncronos de sensado cardíaco para garantizar que no degrade la ventana activa de detección.
- Evaluar el impacto de la distribución de carga y afinidad a núcleos (*core pinning*) entre la tarea de control médico, la supervisión de telemetría y el despachador de servicios de sistema.

2 Investigación previa

1. **Mecanismo de Conmutación de Contexto:** Explique qué ocurre a nivel de procesador durante un *context switch*. Detalle qué información mínima conforma el contexto de una tarea (como el *Program Counter*, puntero de pila y registros de propósito general) y dónde se almacena temporalmente durante la conmutación.
2. **Bloque de Control de Tarea (TCB):** Describa la estructura y el rol del *Task Control Block* (TCB) en FreeRTOS. ¿Cómo utiliza el planificador (*scheduler*) la información del TCB para alternar la ejecución entre tareas en estado *Ready*, *Running* y *Blocked*?
3. **Planificación Apropiativa y Tick de Sistema:** Defina el concepto de *preemptive scheduling* frente a la planificación cooperativa. Explique la función de la interrupción periódica del reloj (*SysTick*) en el disparo de cambios de contexto y la resolución temporal de FreeRTOS (`configTICK_RATE_HZ`).
4. **Sobrecarga y Latencia de Interrupción:** Defina qué es el *overhead* de conmutación de contexto y qué diferencia existe respecto a la latencia de interrupción (*ISR latency*). En un sistema de soporte vital como un marcapasos, ¿por qué es crítico acotar el valor máximo de estos tiempos?
5. **Afinidad de Núcleo (Core Pinning) en SMP:** Explique cómo gestiona FreeRTOS el multiprocesamiento simétrico (SMP) en el ESP32. ¿Qué implicaciones de concurrencia y latencia existen al fijar una tarea médica a un núcleo específico (`xTaskCreatePinnedToCore`) versus permitir balanceo dinámico entre los dos núcleos?

3 Caso de estudio

De: Dirección de Ingeniería y Desarrollo

Para: Equipo de Desarrollo de Firmware RTOS (R&D)

Fecha: 31 de Agosto de 2026, 09:00 CST

Asunto: Requerimientos de arquitectura y concurrencia para el planificador del marcapasos VVI

Hola equipo,

Derivado de la última sesión de alineación con la Dirección de Sistemas Clínicos y la Gerencia de Calidad, hemos definido los criterios de diseño que regirán la implementación del cambio de contexto y la gestión de tareas para la tercera iteración del firmware de soporte vital.

Les reenvío el mensaje con las directrices técnicas acordadas en la junta. Deberán analizar detenidamente el texto para identificar y clasificar las directrices funcionales y operativas.

Saludos cordiales,

Líder Técnico de Proyecto

—— Mensaje Reenviado ——

De: Dirección de Sistemas Clínicos

Para: Dirección de Ingeniería y Desarrollo

Fecha: 28 de Agosto de 2026, 18:40 CST

Asunto: RE: Lineamientos de concurrencia y determinismo temporal en RTOS

Estimado equipo de desarrollo,

Para garantizar la seguridad del paciente bajo nuestra arquitectura de marcapasos VVI sobre ESP32, es imperativo cumplir con las siguientes especificaciones operativas y de sincronización:

1. **Gestión de prioridades y preempción:** El sistema operativo en tiempo real debe garantizar que la tarea de control médico (`vvi_controller_task`) posea una prioridad estrictamente superior a cualquier otra tarea de procesamiento secundario o telemetría, permitiendo la expulsión inmediata de la CPU ante estímulos cardíacos.
2. **Ventanas temporales estrictas:** El ciclo base del marcapasos opera a 60 ppm con un intervalo de escape de 1000 ms y un periodo refractario ventricular (VRP) de 250 ms. La latencia total de respuesta del sistema desde que se genera una interrupción por sensado hasta que la tarea médica procesa la inhibición no debe exceder los 120 μ s.
3. **Aislamiento de núcleos (Core Pinning):** La tarea crítica de control médico debe ejecutarse de manera exclusiva en un núcleo dedicado del microcontrolador (Core 1), mientras que los procesos de diagnóstico o soporte secundario deben aislarse para evitar interferencias de temporización.
4. **Manejo seguro de interrupciones:** Las Rutinas de Servicio de Interrupción (ISR) asociadas al pin de sensado deben limitar estrictamente su labor al intercambio de semáforos o banderas, delegando el procesamiento pesado al planificador mediante llamadas de cambio de contexto explícitas.

Por favor procedan con el análisis de los requerimientos y la documentación de diseño correspondiente.

4 Desarrollo

4.1 Análisis de requerimientos y extracción de restricciones

La capacidad de distinguir entre lo que el sistema *debe hacer* y las condiciones bajo las cuales *debe operar* es fundamental en el diseño de sistemas críticos.

Instrucciones: Diríjase al mensaje reenviado por la Dirección de Sistemas Clínicos en la sección del Caso de Estudio. Utilizando dos colores de marcatextos (o subrayado digital), identifique y clasifique la información del texto de la siguiente manera:

- **Color 1 (Ej. Amarillo) - Requerimientos Funcionales:** Subraye las sentencias que describan acciones, comportamientos específicos o funciones que el código debe ejecutar (ej. encender un pin, enviar un mensaje, cambiar un estado).
- **Color 2 (Ej. Verde) - Requerimientos No Funcionales / Restricciones:** Subraye las sentencias que impongan límites de tiempo, restricciones de arquitectura, condiciones de rendimiento o seguridad (ej. latencias máximas, prioridades relativas, asignación de núcleos).

4.2 Simulación manual del planificador (Traza de ejecución)

Para comprender cómo el núcleo de FreeRTOS administra el procesador, es fundamental interiorizar su máquina de estados. La Figura 1 ilustra los tres estados principales de una tarea en el planificador:

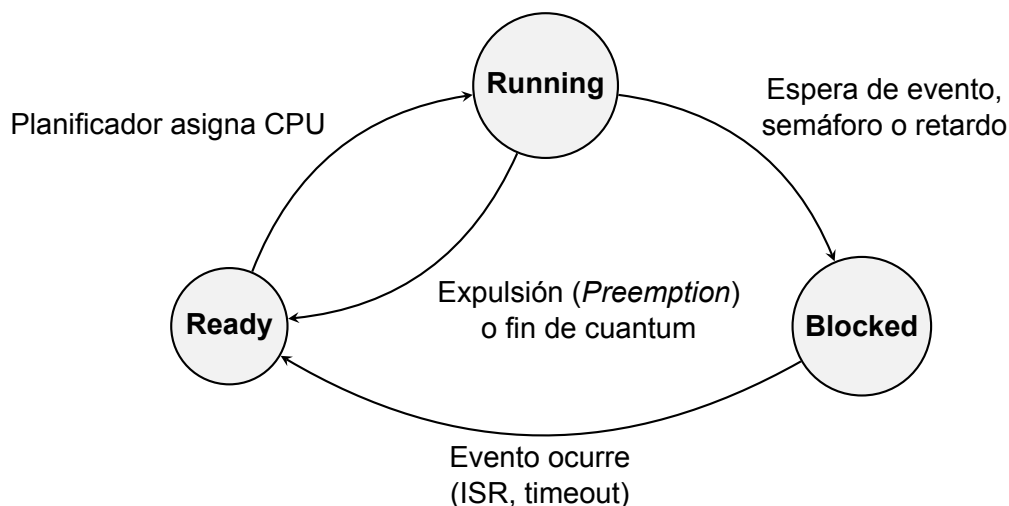


Figura 1: Diagrama simplificado de estados de una tarea en FreeRTOS.

En los siguientes ejercicios analizaremos la interacción entre las tareas de nuestro sistema asumiendo que corren en un **solo núcleo (Core 1)** para forzar la contienda por la CPU. Contamos con las siguientes tareas ordenadas por prioridad:

1. `kernel_task` (Prioridad 4 - Máxima)
2. `vvi_task` (Prioridad 3 - Alta)
3. `diag_task` (Prioridad 1 - Baja)

Ejemplo resuelto: Vencimiento de intervalo de escape

Condición Inicial ($t = 0$ ms):

- `kernel_task`: *Blocked* (Esperando peticiones en `syscall_queue`).
- `vvi_task`: *Blocked* (Esperando onda R o timeout de 1000 ms).
- `diag_task`: *Running* (Ejecutando cálculos continuos en CPU).

Evento: En el instante $t = 1000$ ms, expira el temporizador de la ventana de escape sin que haya ocurrido un latido intrínseco.

Tarea	Estado Pre-vio	Estado Final	Justificación / Mecanismo de OS
<code>vvi_task</code> (Pri 3)	Blocked	Running	El <i>timeout</i> expira. Pasa a <i>Ready</i> . Al tener prioridad 3 (mayor a 1), el planificador le asigna inmediatamente la CPU.
<code>diag_task</code> (Pri 1)	Running	Ready	Sufre una expulsión (preemption) por parte de <code>vvi_task</code> . Su contexto se guarda en el <i>stack</i> y libera la CPU.
<code>kernel_task</code> (Pri 4)	Blocked	Blocked	Ningún evento en la cola de syscalls la ha despertado; permanece inactiva.

Ejercicio 1: Detección asíncrona (Dificultad Media)

Condición Inicial ($t = 0$ ms): Igual que en el ejemplo resuelto (`diag_task` en *Running*, las demás en *Blocked*).

Evento: En el instante $t = 200$ ms, llega un flanco de subida al pin de sensado (GPIO 4), disparando la Interrupción de Hardware (ISR). La ISR libera el semáforo `sense_event_sem` y solicita un cambio de contexto.

Instrucción: Complete el estado final en $t = 200$ ms para las siguientes tareas y justifique.

Tarea	Estado Pre-vio	Estado Final	Justificación / Mecanismo de OS
<code>vvi_task</code> (Pri 3)	Blocked		
<code>diag_task</code> (Pri 1)	Running		

Ejercicio 2: Entrada en periodo refractario (Dificultad Media)

Condición Inicial ($t = 201$ ms): Tras procesar la interrupción anterior, `vvi_task` está en *Running* y acaba de invocar `sys_sleep(250)` para iniciar su periodo refractario.

Evento: El kernel procesa la llamada de retardo.

Instrucción: Complete el estado final en $t = 201$ ms.

Tarea	Estado Pre-vio	Estado Final	Justificación / Mecanismo de OS
<code>vvi_task</code> (Pri 3)	Running		
<code>diag_task</code> (Pri 1)	Ready		

Ejercicio 3: Elevación de privilegios mediante Syscalls (Dificultad Alta)

Condición Inicial ($t = 500$ ms): `vvi_task` sigue en *Blocked* cumpliendo su retardo. `diag_task` está en *Running*. `kernel_task` (Pri 4) se encuentra en *Blocked* esperando comandos en la `syscall_queue`.

Evento: `diag_task` invoca la función `sys_log_event("Prueba")`. Internamente, esta función empaqueta un mensaje y lo envía a la cola `syscall_queue` usando `xQueueSend()`.

Instrucción: Complete el estado final y analice la cadena de eventos. ¿Quién ejecuta realmente el `Serial.printf()`?

Tarea	Estado Pre-vio	Estado Final	Justificación / Mecanismo de OS
<code>kernel_task</code> (Pri 4)	Blocked		
<code>diag_task</code> (Pri 1)	Running		

4.3 Análisis e implementación del sistema

Los archivos fuente completos de esta iteración (`main.cpp`, `syscalls.cpp`, `syscalls.h`, `bootstrap.cpp` y `bootstrap.h`) se encuentran adjuntos en la plataforma virtual del curso. Descargue el proyecto, analice la arquitectura final y ejecútelo en su entorno de desarrollo para observar el comportamiento de las tareas y la concurrencia a través de la terminal serie.

Para demostrar la asimilación técnica del código, elabore un **diagrama de flujo de interacción** que documente la ruta crítica de ejecución. El diagrama debe ilustrar la secuencia exacta de eventos desde que se detecta el flanco de subida en el pin de sensado (GPIO 4), el disparo de la ISR, la liberación del semáforo, hasta el instante en que `vvi_controller_task` es despertada y evalúa la condición de inhibición. Incorpore este diagrama en su reporte.

Posteriormente, con base en la lectura minuciosa de los archivos fuente, responda de forma analítica las siguientes preguntas:

1. ¿Qué función específica de la API de FreeRTOS se encarga de solicitar explícitamente el cambio de contexto desde la rutina de servicio de interrupción (ISR)?
2. En la implementación de la interrupción, ¿cuál es el propósito técnico de la variable `xHigherPriorityTaskWoken` y qué sucedería con el determinismo del marcapasos si se omitiera su evaluación al salir de la ISR?
3. Describa el mecanismo exacto por el cual la llamada a `sys_wait_sensing()` bloquea a la tarea de usuario. ¿Qué primitiva de sincronización se utiliza y qué función del kernel es la responsable de poner la tarea en estado *Blocked*?
4. Al invocar `sys_pace_pulse()`, analice el flujo de datos. ¿Por qué la tarea médica no escribe directamente en el GPIO 5 y cómo viajan los parámetros de la solicitud hacia la tarea `Kernel-Service`?

Evaluación de la Práctica

Instrucciones: Indique su grado de acuerdo con cada una de las siguientes afirmaciones utilizando una escala de 1 a 5, donde:

1. Totalmente en desacuerdo
2. En desacuerdo

3. Neutral
4. De acuerdo
5. Totalmente de acuerdo

Criterio / Afirmación	1	2	3	4	5
Los conceptos y habilidades desarrollados en esta práctica son relevantes para mi formación profesional.					
El trabajo realizado me permitió comprender de forma práctica la teoría revisada en clase.					
Las actividades y problemáticas planteadas durante la sesión resultaron atractivas y motivadoras.					
El enfoque práctico y los retos propuestos captaron mi interés a lo largo del desarrollo.					
El nivel de complejidad técnica fue adecuado respecto a los conocimientos y recursos previos disponibles.					
La claridad de la guía permitió realizar la práctica de manera fluida y sin bloqueos innecesarios.					